`std::bitset` inclusion test methods

Vittorio Romeo | http://vittorioromeo.info

Document number: **P0125R0**

Table of contents

Introduction

The `std::bitset` class provided by the standard library can be used to model the mathematical "set" structure, which represents a collection of distinct objects.

- Every bit represents a distinct object.
- If the *nth* bit is `1`, the *nth* object is present in the set.
- Otherwise, the *nth* object is not present in the set.

Existing operators and methods allow us to perform common set operations:

- The `&` "bitwise and" operator represents intersection.
- The `|` "bitwise or" operator represents union.
- The `^` "bitwise xor" operator represents symmetric difference.

Two missing set operations that can be very useful are related to the concept of "inclusion":

- If every object contained in set A is also contained in set B, then A is said to be a subset of B.
- If A is a subset of B, or if A is equal to B, then B is a superset of A.
- If A is a subset of B, but A is not equal to B, then A is a proper (or strict) subset of B.
- If A is a subset of B, but A is not equal to B, then B is a proper (or strict) superset of A.

Checking if a bitset is a subset of another is currently possible by combining existing operators - unfortunately this solution does not guarantee possible short-circuiting optimizations and isn't ideal in terms of code clarity and readability.

```
// Current solution:
std::bitset<256> a, b;
auto a_includes_b(a & b == b);
```

If the bitsets are multi-word, it is not guaranteed that the code above will be optimized to return `false` as early as possible if one of the words does not match (by short-circuiting).

This document proposes the addition of two new methods to `std::bitset`:

- `bitset::is_subset_of(const bitset&)`
- `bitset::is_superset_of(const bitset&)`

The names are self-explanatory, and implementations could explicitly short-circuit for optimal performance.

```
// Proposed solution:
std::bitset<256> a, b;
auto a_includes_b(a.is_subset_of(b));
```

Motivation and scope

One of the strongest points about C++ is the possibility of having **cost-free abstractions**.

- Abstractions make code easier to write and read.
- Cost-free abstractions require no additional runtime overhead.

The proposal of adding a dedicated inclusion test method to `std::bitset` builds upon both these benefits:

- The added abstraction of a method which closely follows the mathematical operation is clear and intuitive.
- Potentially short-circuiting the computation reduces unnecessary runtime overhead.

Impact on the Standard

This proposal is a pure extension on `std::bitset`'s interface. No core language changes are required, no additional library modules are required.

Design decisions and future considerations

The first design decision is the name of the method. Names such as `contains` or `includes` have been considered, but, in accordance with N2050, `is_subset_of` is definitely the best alternative, as it explains as intuitively as possible the operation that's being executed and closely follows the mathematical language of sets and subsets.

Another decision regards the signature of the method and how bitsets of different sizes would interact. One way of dealing with the issue, that would also maintain consistency with existing `std::bitset` binary operations (`&` , `|` , `^`), would be allowing calls to `is_subset_of` only between `std::bitset` with the same size.

Following N2050's original ideas, the addition of `is_subset_of` can later be expanded adding an additional argument for an offset, or with overloads for bitsets with different size. Having an offset argument would allow meaningful set operations between bitsets of different sizes.

```
namespace std {
template<size_t N> class bitset {
    // ...

    // (Currently proposed.)
    // Inclusion checking between bitsets of the same size:
    bool is_subset_of(const bitset<N>& rhs) const noexcept;

    // (Possible future proposal.)
    // Inclusion checking between bitsets of different size:
    template<std::size_t K>
    bool is_subset_of(const bitset<K>& rhs, std::size_t offset = 0) const noexcept;

    // ...
};
}
```

Another possible future addition is the “proper” version of the proposed methods:

- `bitset::is_proper_subset_of(const bitset&)`
- `bitset::is_proper_superset_of(const bitset&)`

The above “proper” methods would return `false` if the bitsets were equal.

Proposed wording

Add to `<bitset>` header:

```
namespace std {
template<size_t N> class bitset {
public:

    // ...

    bitset<N>& operator&=(const bitset<N>& rhs) noexcept;
    bitset<N>& operator|=(const bitset<N>& rhs) noexcept;
    bitset<N>& operator^=(const bitset<N>& rhs) noexcept;

    // ...

+++ bool is_subset_of(const bitset<N>& rhs) const noexcept;
+++ bool is_superset_of(const bitset<N>& rhs) const noexcept;

    // ...
};
}
```

Example implementation

```
namespace std {

template<size_t N>
bool bitset<N>::is_subset_of(const bitset<N>& rhs) const noexcept {
    for (auto i = 0; i < bitset<N>::number_of_words; ++i)
        if (rhs.words[i] & ~(words[i]) != 0)
            return false;

    return true;
}

template<size_t N>
bool bitset<N>::is_superset_of(const bitset<N>& rhs) const noexcept {
    return rhs.is_subset_of(*this);
}

}
```

References

- N2050
- Original std-proposals thread
- StackOverflow question #19258598